

Task- and dataset-specific information in protein language models

Roman Joeres^{*1,2}, Ilya Senatorov^{*1,2}, Anastasia Kolchina^{1,2,3}, Dietrich Klakow^{2,3,4}, and Olga V. Kalinina^{†1,2,4,5}

¹Drug Bioinformatics, Helmholtz Institute for Pharmaceutical Research Saarland, 66123 Saarbruecken, Germany

²Center for Bioinformatics, Saarland University, 66123 Saarbruecken, Germany

³Saarland Informatics Campus, Saarland University, 66123 Saarbruecken, Germany

⁴Pharma Science Hub, 66123 Saarbruecken, Germany

⁵Medical Faculty, University Hospital Saarland, 66421 Homburg, Germany

Abstract

Protein language models (PLMs) have transferred the latest advances from natural language processing to computational biology. These models, trained on large corpora of protein sequence data, are widely used to translate amino acid sequences into latent-space embeddings, ready for use in diverse downstream tasks (DTs). By a common consensus, embeddings from the model’s last layer are used, and the model’s internal behavior remains poorly understood. We analyzed 13 PLMs across 15 DTs from 11 datasets to investigate the informativeness of embeddings created in intermediate PLM layers. We trained probe models on embeddings from each layer, compared their performance, and computed characteristics of the latent spaces they span to estimate the information they contain, and found that the last layers of PLMs rarely contained embeddings that led to the best results on downstream tasks. Furthermore, we identified a connection between DTs and the distribution across PLMs’ layers of the relevant information to predict that task. For example, similarity between the pre-training objective and the objective of predicting properties of individual residues leads to a steady increase in understanding of such tasks across the layers of PLMs. On the other hand, for whole-protein tasks, we observe that the dataset, rather than the task itself, defines PLMs’ ability to perform well on a DT. Embeddings from shallow layers of PLMs perform better for datasets that contain deep mutational scan (DMS) data, while datasets containing diverse natural proteins find most useful embeddings in the models’ deeper layers. Additionally, we discover that the performance of PLMs drops significantly when tasks are introduced for artificial proteins.

Keywords: Protein Language Model, Probing, Protein Property Prediction

^{*}R.J. and I.S. contributed equally to this work.

[†]Corresponding author: olga.kalinina@helmholtz-hips.de

1 Introduction

Since the release of ESM-1 in 2019 [1], the first transformer-based protein language model (PLM), there has been a steady rise in their popularity. PLMs are often used as crucial components in feature engineering processes for predicting protein properties and functions, such as enzymatic activity, binding-site identification, and antibody design [2, 3, 4, 5]. However, it is still not well understood how PLMs represent proteins internally, what information they extract, where across the layers they store it, and whether this varies with the type of protein.

It is generally accepted that the layers in deep learning (DL) models compose abstractions within their internal representations to incrementally build more informative embeddings, leading to the last layer being “the best” in a DL model. This idea is supported by Alain & Bengio [6], who introduced *linear classifier probes* to investigate how informative, with respect to a downstream application, also called a *downstream task* (DT), each layer of a deep neural network is. For the computer vision model ResNet-50 [7], they found that the prediction error decreases steadily with each deeper layer of the network. This showed that the latent space of deeper layers becomes more linearly separable, which is intuitive, since the model is trained end-to-end on the ImageNet [8] dataset and the last layer is a linear classifier over the final embedding. Therefore, the model naturally strives to improve linear separability with each layer. Analogously, the last layer of a PLM has also been assumed to be the most informative and therefore used to provide embeddings for downstream tasks.

Pre-trained language models, including PLMs, are trained differently: they usually undergo self-supervised pretraining to gain a general understanding of the data. For example, ESM-1 [1] is a masked language model (MLM): its objective during pre-training is to predict masked amino acids in a protein sequence. These generalist models are then fine-tuned or used to generate features for prediction heads on DTs with different objectives [9]. Since they are not trained end-to-end on a particular DT, the assumption that the last layer is the most informative does not necessarily hold. Moreover, the pre-training objective in fact shapes the geometry of the latent space: several studies indicate that bidirectionally trained models

(e.g., via MLM loss) develop differently structured embedding spaces compared to autoregressive models [10, 11, 12, 13]. For PLMs, this distinction is particularly critical due to a scale mismatch: while pre-training typically operates locally at the token level (individual amino acids), many downstream tasks require global, sequence-level predictions (whole proteins). Consequently, the pre-training task improves understanding at the amino acid level, but not necessarily at the whole-protein level.

Valeriani et al. [10] investigated language models through a different lens: instead of focusing on linear probes, they examined how the embedding spaces change in each layer of the pretrained models. Two families of language models were the main subjects of this research: iGPT [14] and ESM-2 [15], which were applied to DTs of image analysis and homology detection, respectively. To investigate changes in embeddings in intermediate layers, they computed the intrinsic dimension (ID) of the latent spaces and the neighborhood overlap, i.e., the number of neighbors a sample shares across consecutive latent spaces. They found that the latent spaces of the shallowest and deepest layers exhibit similar characteristics. The authors attribute this phenomenon to the MLM loss used during pre-training: in the first layers, the models encode the input tokens, while in the final layer, they produce embeddings to predict masked tokens; therefore, the internal representation of the last layer is similar to that of the input layer, while the intermediate layers can drift in their representation.

There is additional evidence suggesting that intermediate PLM layers may contain valuable information for DTs. For example, Kumar & Jha showed that using the mean of representations from mid-to-final layers’ embeddings yields a more informative protein embedding than the final layer alone in a kinase function prediction DT [16]. Along the same lines, Vig et al. showed that the shallow layers of PLMs learn simpler biophysical properties, such as amino acid features and secondary structures, while deeper layers capture more complex properties, such as binding sites and contact maps [11]. This is consistent with the behavior of neural networks in computer vision, where kernels in shallow layers learn simple concepts like lines and curves, while kernels in deeper layers learn to identify more complex features, such as faces and objects [17]. However, the lack of a sys-

tematic analysis across many PLMs and DTs does not allow us to draw conclusive lessons from these observations.

Most recently, Prabakaran and Bromberg [18] addressed a related topic: how do PLM-based embeddings of natural proteins differ from those of non-natural randomized sequences? They quantify the uncertainty of a protein embedding by the fraction of randomized non-biological sequences among its latent-space neighbors, finding that some embeddings are no more informative than those of random sequences. This measure relies only on embeddings from one layer and leaves an open question: how does the distinction between natural and artificially designed proteins change across a network’s layers?

In this study, we address these gaps by combining two methods — probing and latent-space analysis — and applying them across a variety of PLMs and DTs. We analyze 13 models from five model families across four LLM architectures and 11 datasets, spanning 15 DTs, to develop a well-founded understanding of PLM layer behavior. We show that model performance varies across internal layers’ embeddings, the performance trends differ across DTs, and propose an explanation based on the DTs’ objectives and data. Moreover, we find a strong connection between the embedding informativeness and the dataset used in the DT, differentiating between narrow deep-mutational-scanning datasets and diverse multi-protein datasets. We demonstrate a markedly weaker performance of PLMs for artificially generated proteins, even if they are verifiably functional, and draw implications for the nature of representations learned by the PLMs.

2 Results

We used 13 protein language models (PLMs) to compute protein-level embeddings for 10 datasets [19, 20, 21, 22, 23, 24, 25, 26, 27] and residue embeddings for two [27, 28]. These datasets correspond to the following 15 DTs: fluorescence intensity regression and active/inactive classification on the fluorescence dataset [19], mutational effect regression on the GB1 dataset [20], stability score regression from the Rocklin and Tsuboyama stability datasets [21, 22], melting temperature T_m regression and species classi-

fication from the Meltome Atlas [24], 10-class multi-label classification of proteins into subcellular locations and binary membrane protein classification on DeepLoc2.0 [26], binary solubility classification from DeepSol [25], fold and superfamily classification on proteins as well as 3-class and 8-class secondary structure prediction on residues from SCOPe40 2.08 [27], secondary structure elements were assigned using the DSSP algorithm [29], and binary classification of residues w.r.t. small-molecule interactions from Littmann et al. [28] (see Section 4.3 for more details).

The 13 PLMs comprise 5 ESM-2 models [15] (ESM-2 8M, ESM-2 35M, ESM-2 150M, ESM-2 650M, and ESM-2 3B), ESMC-300m and ESMC-600m [30], ProtT5 [31], ProstT5 [32], ProGen2-small, ProGen2-medium, and ProGen2-large [33], as well as ProtGPT2 [34]. The first 9 of them are trained bidirectionally; the last 4 are trained with an autoregressive loss, and ProtGPT2 additionally used a BPE tokenizer for subword tokenization [35] (see Section 4.2 for details).

These PLMs are based on four different language model architectures. The original *transformer* architecture embeds the self-attention mechanism into an encoder-decoder architecture, originally designed for language translation tasks [36]. *BERT* extracted the encoder of the transformer architecture to use it bidirectionally, optimizing it for sequence- and token-level understanding through feature extraction or task-specific fine-tuning [9]. The *T5* architecture returns to the full encoder-decoder architecture and is designed to unify all NLP tasks by framing them as “text-to-text” tasks [37]. Finally, the *generative pre-trained transformer*, GPT, is a unidirectionally trained architecture for token generation [38].

2.1 The deepest layer is not always the most informative one

We assessed the performance of PLMs using linear probes (Figure 1), and only in a few cases did we find that the embeddings of the deepest layer yielded the best performance in the corresponding DT (17.92% in Figure 1A). For most PLMs and protein-level DTs, performance increases dramatically over the first few layers, peaks between the 10th and 90th percentiles of model depth, and then declines in the deepest layers (Figure 1B and E). This behavior is consistent

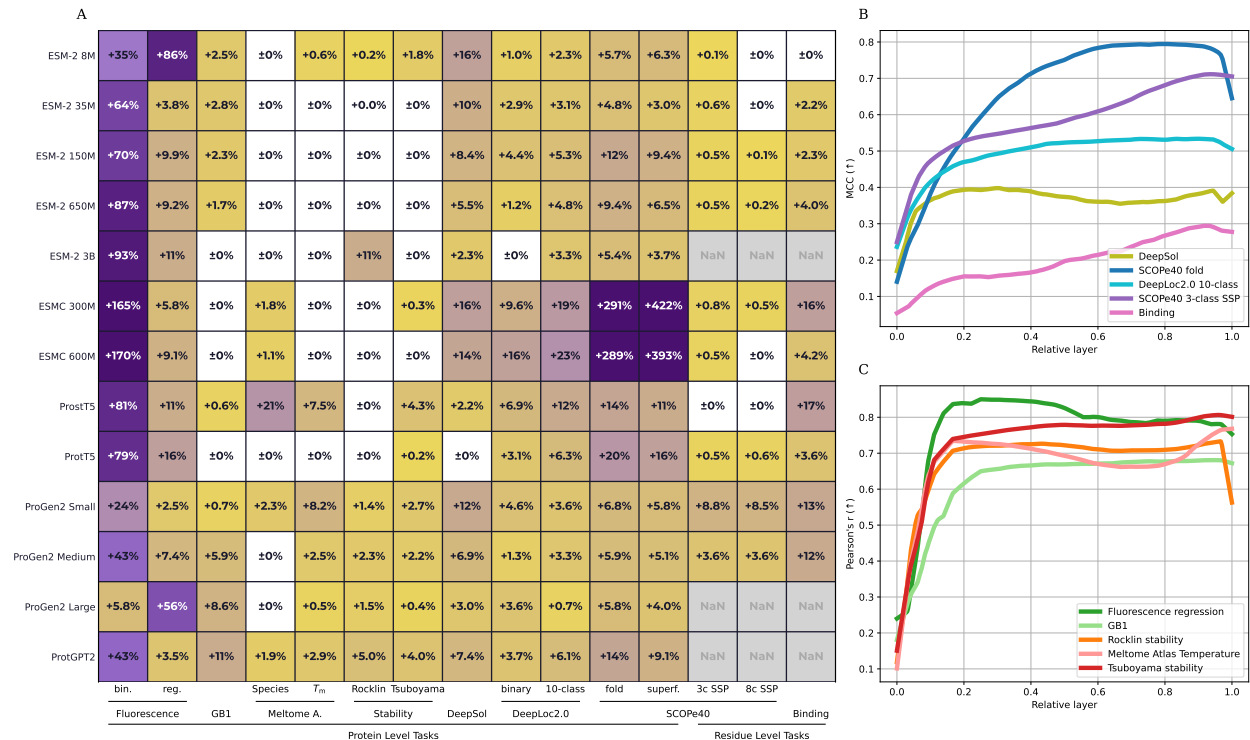


Figure 1: Performance Overview. **A:** Relative improvement of the best layer of each PLM over the last layer on all DTs. **B and C:** Average performance for each dataset across all models for regression (B) and classification (C) tasks. We show the mean over the 13 PLMs. Supplementary Figure 1 provides a visualization with all PLMs.

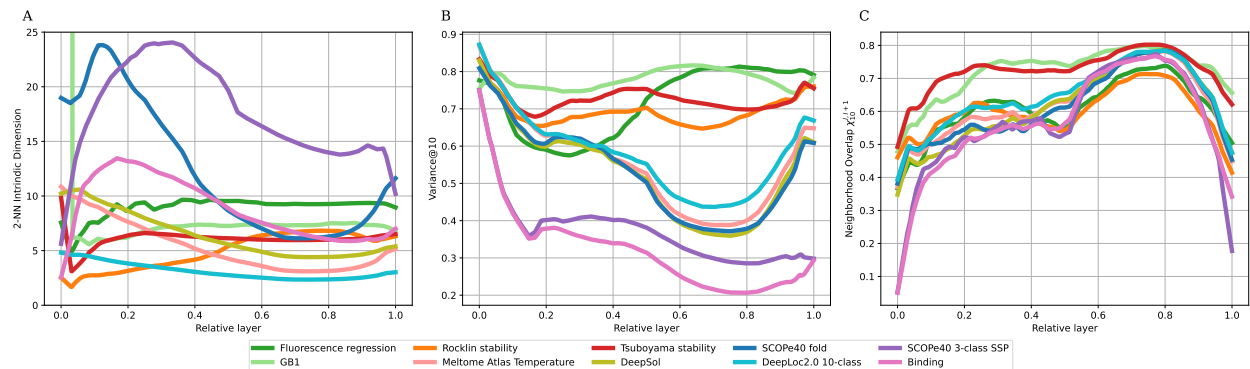


Figure 2: Changes of latent spaces spanned by individual layers of PLMs. The metrics are defined in the Methods section (rows: Intrinsic Dimension, Neighborhood Overlap, and Explained Variance by the first 10 principal components) and evaluated on the latent spaces spanned by the 13 PLMs. Supplementary Figures 2 and 3 visualize these metrics for the individual PLMs.

across different PLMs, datasets, and DTs. Moderate exceptions to that are generative models (ProGen2 and ProtGPT2), which experience a smaller drop-off in the deepest layers than MLM models. Interestingly, this differs for the residue-level DTs: there, most models show a steady increase in performance with each

Table 1: Comparison of the performances of linear probes of the best layers of all PLMs and all tasks.

Models	PROTEIN-LEVEL TASKS										RESIDUE-LEVEL TASKS				
	Fluorescence		GB1	Meltome Atlas		Stability		DeepSol	DeepLoc2.0		SCOPe40	SSP		Binding	
	Bin.	Reg.		T_m	Species	Rocklin	Tsubo.		Bin.	10c		Fold	SF.		3c
ESM-2 8M	0.583	0.690	0.850	0.434	0.711	0.726	0.427	0.294	0.657	0.488	0.693	0.753	0.649	0.534	0.475
ESM-2 35M	0.614	0.711	0.854	0.472	0.715	0.740	0.565	0.320	0.675	0.537	0.796	0.867	0.703	0.585	0.475
ESM-2 150M	0.613	0.715	0.866	0.573	0.778	0.744	0.660	0.369	0.689	0.578	0.837	0.901	0.737	0.621	0.489
ESM-2 650M	0.621	0.722	0.879	0.622	0.782	0.747	0.715	0.377	0.695	0.580	0.834	0.893	0.753	0.640	0.515
ESM-2 3B	0.619	0.717	0.904	0.628	0.779	0.746	0.737	0.370	0.697	0.563	0.817	0.880	nan	nan	nan
ESMC 300M	0.647	0.735	0.867	0.704	0.850	0.749	0.734	0.366	0.700	0.576	0.817	0.870	0.745	0.626	0.466
ESMC 600M	0.637	0.724	0.873	0.750	0.863	0.752	0.759	0.363	0.693	0.576	0.806	0.881	0.749	0.629	0.464
ProtT5	0.621	0.714	0.872	0.711	0.839	0.756	0.643	0.406	0.702	0.582	0.795	0.871	0.760	0.641	0.475
ProstT5	0.630	0.721	0.886	0.446	0.751	0.753	0.723	0.350	0.682	0.537	0.789	0.860	0.825	0.727	0.523
ProGen2 Small	0.651	0.738	0.678	0.464	0.775	0.718	0.546	0.321	0.651	0.488	0.673	0.737	0.487	0.407	0.339
ProGen2 Medium	0.652	0.739	0.776	0.587	0.775	0.731	0.626	0.371	0.660	0.515	0.729	0.802	0.555	0.464	0.365
ProGen2 Large	0.741	0.791	0.746	0.637	0.797	0.725	0.692	0.363	0.665	0.541	0.683	0.749	nan	nan	nan
ProtGPT2	0.601	0.681	0.785	0.499	0.728	0.691	0.556	0.332	0.656	0.525	0.717	0.785	nan	nan	nan

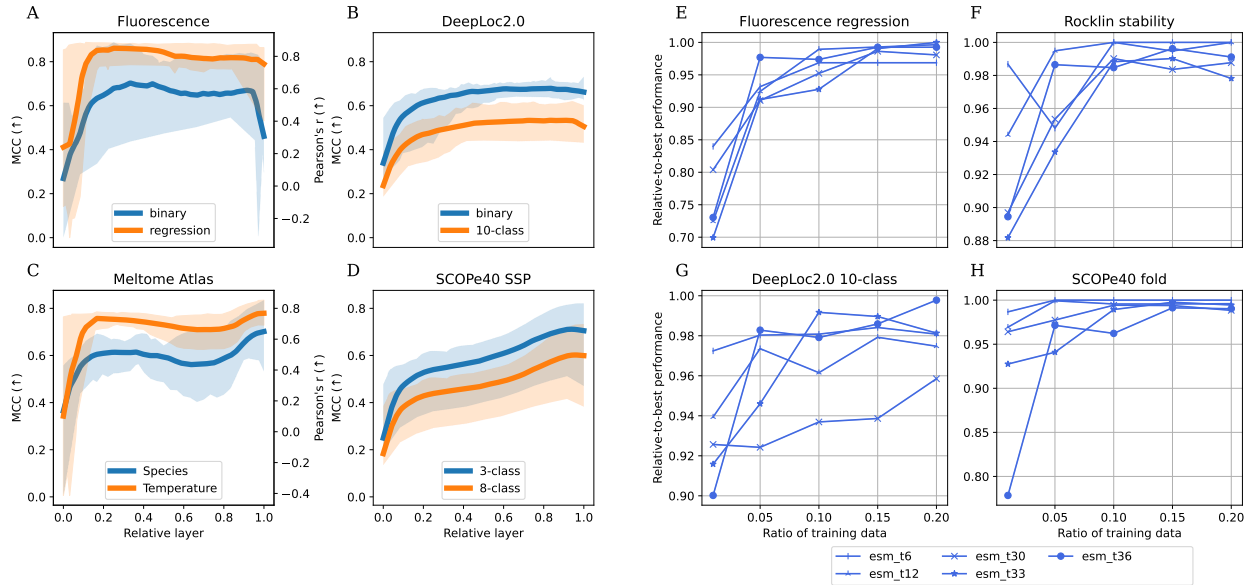


Figure 3: **Ablation studies.** **A-D:** Pairwise comparisons of two tasks for one dataset, showing that model performance patterns do not change across DTs within the same model and dataset. **E-H:** Subsampling of the DT training dataset shows that relatively little data is sufficient to find a PLM layer that achieves close-to-optimal performance. These experiments were computed over three seeds; in each of them, the same layer was identified as “the best”, showing the robustness of testing on subsets.

deeper layer (Figure 3C and D and Supplementary Figures 2 and 3).

This non-linear, non-monotonic behavior of consecutive layers on protein-level DTs is also reflected in metrics of the latent spaces spanned by the individual layers (Figure 2). We calculate three metrics for the latent spaces, capturing their complexity and change across the layers: (i) the *intrinsic dimension* (ID) describes how many independent variables (dimensions)

are necessary to accurately describe a vector space. To calculate the ID, we use the TwoNN approach [39].

(ii) The *neighborhood overlap* measures the mean ratio of neighbors a sample shares between latent spaces in two given layers of the model (the higher the overlap, the more similar the latent spaces are) [40], and (iii) the *variance@10*, which measures the fraction of variance in the latent space explained by the first 10 principal components [41] (see Section 4.1.1 for more

details). The behavior of these metrics reflects the models’ performances (Figure 1B & C): the curves approach a similar range of values at the end, as they did at the beginning, indicating that the latent spaces are shaped to predict masked tokens again, confirming the findings of Valeriani et al. [10].

This behavior of latent-space informativeness is independent of factors such as data split, objective, and task difficulty. We use the number of classes as a proxy for perceived difficulty; the more classes a task has, the more difficult it is. Furthermore, we consider regression more difficult than classification. We demonstrate this on four datasets (fluorescence, DeepLoc2.0, the Meltome Atlas, and secondary structure prediction) by comparing two tasks for each, differing in split, objective, and perceived difficulty based on their intrinsic label hierarchy (Figure 3A-D). For the fluorescence dataset, the tasks differ in the objective (a classification task into active and inactive fluorescing proteins and a regression task predicting the raw log-scaled fluorescence intensity values). For the DeepLoc2.0 dataset, we consider binary (membrane or non-membrane) and 10-class (based on subcellular localization information) classification tasks. The Meltome Atlas offers the two tasks of T_m regression and species classification, which are closely related as shown by Lopez et al. [42]. Similarly to DeepLoc2.0, for the secondary structure prediction (SSP), we consider three- and eight-class classification tasks. The models’ performance behavior for all three pairs of tasks is similar across all layers: three out of four mean Pearson correlation coefficients between the performance curves of the individual models for the datasets are ~ 0.85 or above (Fluorescence: 0.681 ± 0.241 , Meltome Atlas: 0.847 ± 0.103 , DeepLoc2.0: 0.978 ± 0.024 , SCOPe40 SSP: 0.996 ± 0.003), indicating that the very same layers of the corresponding PLMs produce embeddings yielding highly similar performance in both task variants in all four datasets, and this is consistent across different splits, objectives, and difficulties.

From a practical perspective, we were interested to find out how fast one can identify the best layer of a PLM for a given DT. To investigate that, we uniformly downsampled the full dataset and retained the data splits for all datasets except SCOPe40, for which we computed stratified splits using DataSAIL [43] to ensure that members of each class were present

in each split. We tested the datasets with the ESM-2 models and found that just 15-20% of the data is sufficient to identify a layer achieving at least 95% of the best performance (Figure 3E-H; Supplementary Figure 3 for all datasets). Additionally, we observed that larger models are more robust to sparse data in the DTs (Figure 3E-H).

2.2 End-to-end vs. piecewise training

In the introduction, we hypothesize that when a model is trained end-to-end on a specific task, each layer contributes to solving that task, and layers sequentially become better at predicting that task. In contrast, when training is done piecewise, i.e., first pretraining a model, then training a downstream prediction head on its embeddings, layers might not sequentially improve at predicting the downstream task. To test this hypothesis, we conduct three experiments: (i) evaluate the layers of PLMs on their pretraining objective, (ii) fine-tune PLMs on DTs and analyze the performance of individual layers on the DTs, and (iii) take those fine-tuned PLMs and evaluate their performance on the pretraining objective again. We conducted the first experiment on the ESM-2 model family with the MLM loss and on the ProGen2 model family with the next token prediction (NTP) loss. Therefore, we sampled 10,000 sequences uniformly from the SCOPe40 dataset and masked 15% of the residues for the MLM loss dataset. For the NTP loss dataset, we uniformly sampled a position in each of the 10,000 sequences, split the sequence at that position, and used the next token as the label. Then, we embedded the amino acids using the ESM-2 and ProGen2 models and fitted a linear probe to predict the amino acid type from the embeddings of the masked residues and the next token from the last residue embedding of the cut sequences, respectively. The pre-training losses, objectives for which the ESM-2 and ProGen2 models have been end-to-end optimized, decrease almost monotonically with each deeper layer (Figure 4A), meaning that PLMs improve on their training objectives monotonically with each layer, with informativeness for DTs being dependent on the individual DT’s similarity to the pre-training objective.

For the second experiment, we fine-tuned the ESM-2 150M model on six DTs, trained linear probes on the fine-tuned ESM-2 layers, and compared their perfor-

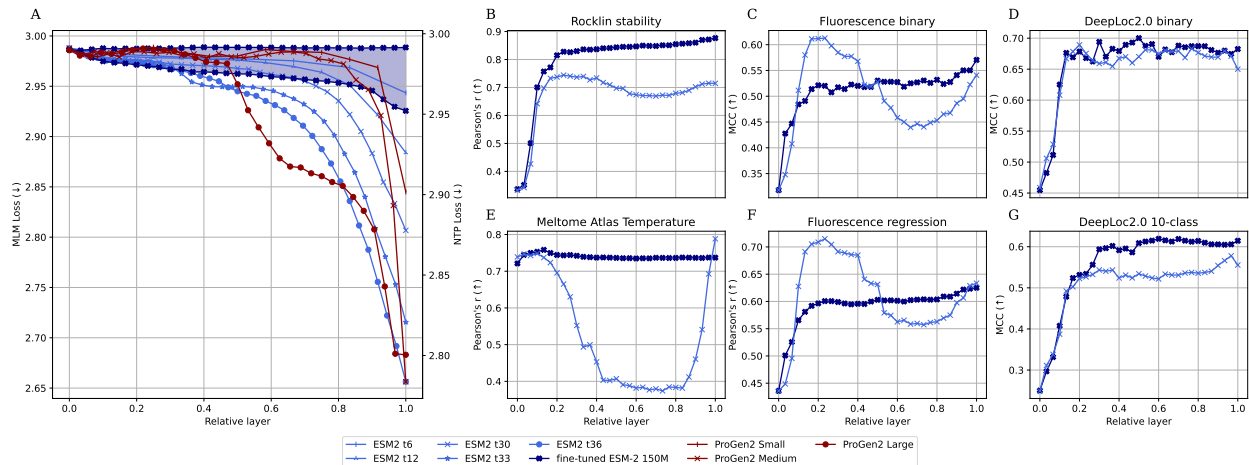


Figure 4: The effect of fine-tuning on model performance. Subfigure A shows that native PLMs improve in each layer on their pretraining objective. This is true for both the MLM objective (ESM-2 models) and the NTP objective (ProGen2 models). It also shows that fine-tuning the PLM worsens the performance on the pretraining objective (shaded area for fine-tuned ESM-2 150M). B-G: On the other hand, fine-tuning leads to each layer improving over its previous layer in predictiveness towards the fine-tuning objective.

mance to that of linear probes trained on the original ESM-2 models (Figure 4B-G). The fine-tuning shows a clear effect, and the performance of the fine-tuned models generally increases monotonically, indicating that the non-monotonic behaviors in Figure 1 can be explained by the training objective of the prediction model.

The third experiment evaluated the MLM loss for each layer of the fine-tuned ESM-2 150M models. Using the MLM loss dataset from the first experiment, we calculated the performance of the six models (shaded area in Figure 4A). Fine-tuning shifts the model’s focus in each layer away from the pre-training objective and toward the DT (Figure 4).

These experiments show that a PLM that is pre-trained on a residue-level objective improves in each layer with respect to residue-level DTs (Figure 3C and Figure 4A). But when it is applied to protein-level tasks, the last layer is not the best (Figure 4B-G). When the PLM is then fine-tuned on a protein-level task, it improves in each layer with respect to protein-level tasks (Figure 4B-G), but drops in residue-level performance (Figure 4A). This observation is supported by the fact that performance steadily increases for residue-level tasks, where the objective by design aligns better with the pre-training objective that is

also residue-level.

2.3 Performance trends over PLM layers across different datasets

Further, we analyzed PLMs’ performance for different datasets and protein-level DTs (Figures 1 and 2 and Supplementary Figures 2 and 3). The performance-over-layer curves for the Fluorescence and GB1 datasets differ from those of the DeepLoc2.0, DeepSol, and protein-level SCOPe40 datasets: for most models, the fitness prediction performance peaks with embeddings from shallow layers and then decreases significantly in the deeper half of the model, whereas the performance for the DeepLoc2.0, DeepSol, and SCOPe40 datasets increases steadily, then declines over the deepest 5% of the model, most probably due to the MLM loss as explained in Section 2.2. The intrinsic dimension and variance@10 metrics also differ clearly: they increase for the Fluorescence and GB1 datasets, and decrease for the DeepLoc2.0, DeepSol, and the protein-level SCOPe40 datasets as layer depth increases. For residue-level tasks, performance always increases with the layer depth.

We investigated how this behavior may correlate with the nature of the proteins comprising the datasets:

in the Fluorescence dataset, we have 54,024 variants of a single protein, GFP, from *A. victoria* created via deep mutational scanning (DMS) and harboring at most 16 mutations, and the GB1 dataset contains a DMS of the IgG-binding domain of protein G, whereas the DeepLoc2.0, DeepSol, and SCOPe40 datasets are comprised of many proteins with essentially no sequence identity covering broader parts of the protein space (Table 3).

The Rocklin stability dataset [21] sits in between: it comprises four datasets that include both DMS variants of 14 artificial and 3 natural proteins (with 631 to 829 mutants per protein) and diverse artificial proteins created using a physics-based Rosetta pipeline (cf. Section 4.3), and may therefore exhibit mixed and smoothed characteristics (Figures 1 and 2).

Notably, PLM embeddings perform much worse on the artificial proteins of the Rocklin stability dataset than on datasets composed of natural proteins or their close mutants (Figure 5A). To investigate that, we considered another dataset of 90 artificially generated and 90 natural lysozyme sequences, where the generation was performed with ProGen v1 [23], and their catalytic activity was measured (relative to hen egg white lysozyme, ranging from -0.077 to 2.535 for artificial and from -0.117 to 0.395 for natural ones) [23]. We similarly trained prediction probes on each layer; for the artificial sequences, PLMs perform generally poorly across all layers showing a very weak improvement towards later layers (Spearman correlation of ~ 0.2 , with an exception for ProtGPT2 which is an outlier in many experiments), despite the fact that the artificial activities vary over a wider range than the natural ones, and for natural sequences most PLMs show a weak trend to improving towards the later layers with a top performance around a Spearman correlation of ~ 0.6 for the best model (Figure 5B). Taken together, these results indicate that PLMs are not very well suited to predict the phenotype for artificial sequences in the two tested cases (stability and lysozyme activity), regardless of the nature of the phenotype and distribution of the proteins in the sequence space.

3 Discussion

In this work, we present the first comprehensive study of the performance of PLM layers across architectures and tasks, providing insights into what information PLMs encode and where in the network this information is stored. First, in agreement with previous work [10, 16, 11], we demonstrated that for most DTs and most PLMs, the last layer does not yield the embeddings best suited to the DT. However, it largely depends on the dataset where in the PLM the best-performing PLM layer for a given DT is situated. The best-performing layer signifies when in the forward pass the model extracts the most relevant information for a particular DT, and this information is diluted again in deeper layers due to the difference between the pre-training and downstream objectives. Importantly, we show that the different informativeness of PLM layers is independent of the data split, objective, and difficulty. Instead, two other factors influence which layers perform better for different DTs.

The first factor is the difference between the pre-training and the downstream objectives. For residue-level tasks, the difference is lower than for protein-level ones, so PLMs tend to improve monotonically on such tasks. For protein-level tasks, the downstream objective does not align so well with the pre-training objective, and the performance behavior across layers can be seen as a sign of how good this alignment is. For DMS-based DTs, this alignment seems to be the worst. Naturally, when models are fine-tuned with an additional downstream objective, their performance monotonically increases for all datasets and DTs.

The second important factor appears to be the dataset structure. The most striking difference is between DMS-based datasets that are centered around one or several proteins and comprise many variants of them (Fluorescence, GB1, DMS part of the Rocklin stability dataset, the Tsuboyama stability dataset) and datasets that cover a large number of diverse proteins (DeepLoc2.0, DeepSol, and the protein-level SCOPe40, Figure 1). For the DMS-based datasets, essentially all PLMs extract most useful information in early layers, whereas for diverse datasets the usefulness of information increases steadily across the layers. One can interpret that as the early layers learn about fine details of the proteins and local contexts, whereas later layers learn more general rules that dif-

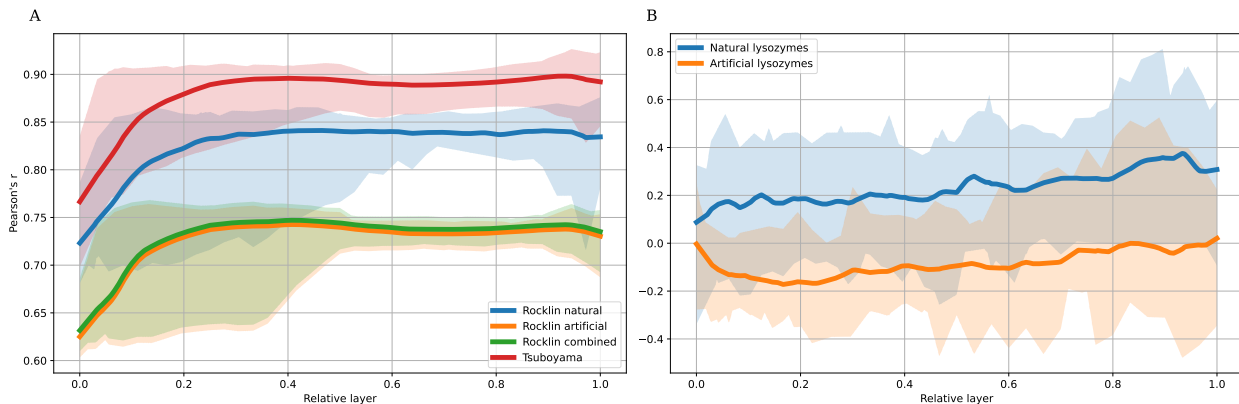


Figure 5: **Artificial protein sets** **A** shows the comparison of the composition of the Rocklin stability dataset into performance on the DMS and the designed proteins. Importantly, we did not train separate models but only one and evaluated this on the DMS and artificial sequences separately. **B** shows the performance differences between predicting properties of lysozymes generated by ProGen v1 versus natural ones. **C** shows the performance of layers on a DeepSol50 set.

ferentiate proteins from one another. This resembles the observation that conventional LLMs extract syntax in early layers and semantics in later layers of the model [44].

Our experiments with artificial proteins (Rosetta-generated part of the stability datasets and ProGen-generated lysozyme sequences) allow us to cautiously draw a broader conclusion: what PLMs accumulate through their layers is not an abstraction of whole-protein function, but of naturally evolved sequence space. It must be noted that PLMs can still meaningfully interpret local, mutational effects even in designed sequences (as can be seen for the DMS datasets of artificial proteins), but they are markedly less capable of predicting protein-level effects (e.g., stability of a new design compared to others or its functional competence). This observation holds independent of the generation engine, both for physics-based Rosetta and for PLM-based lysozyme designs.

To summarize, this work provides insight into how knowledge about proteins is acquired and transformed in PLMs. We show that not only is the last layer almost never the best layer to use in a DT (as it is still conventionally done), but more precisely that PLMs learn fine details about individual proteins in early layers and, with increased influence from more distant residues due to more stacked attention layers, learn general protein properties in deeper layers. Our

analysis shows that PLMs can separate fine-grained and large-scale information about proteins and provides insights into how this information is encoded in their inner layers. It also provides a practical guide to using these inner layers in downstream applications.

4 Methods

4.1 Mathematical foundations

Protein language models (PLMs) follow mostly the same principles as large language models in natural language processing. Input is a sequence of amino acids $(a_i)_{i=1}^m$. For most PLMs, each amino acid corresponds to a token, $t_j = a_i$, but ProtGPT2 utilizes a byte-pair encoding (BPE) tokenizer [35] grouping multiple amino acids into one token, $t_j = (a_k \dots a_l)$. The resulting sequence of tokens $(t_j)_{j=0}^n$ is then processed by an L -layered PLM encoder yielding the token-wise layer activations $\mathbf{h}_{i,j}^l$, representing token t_j in protein i of layer l ¹. For the remainder of this section, we assume a token corresponds to one amino acid and use the terms interchangeably in this context.

¹ProtGPT2 cannot be applied to amino acid-level prediction tasks because its token embeddings represent multiple amino acids due to BPE tokenization. Therefore, ProtGPT2 only produces whole-protein embeddings.

For whole-protein prediction tasks, we use mean pooling to compute per-layer embeddings $\mathbf{h}_i^l$ of a protein i as:

$$\mathbf{h}_i^l = \frac{1}{n} \sum_{j=0}^n \mathbf{h}_{i,j}^l.$$

We call the vector space $\mathcal{H}^l$ over all protein embeddings $\mathbf{h}_i^l$ the *layer-latent space* for a dataset and the specific layer l of a PLM.

4.1.1 Latent space metrics

We use three metrics to analyze the complexity and structure of $\mathcal{H}^l$ and the changes between $\mathcal{H}^l$ and $\mathcal{H}^{l+1}$. The intrinsic dimension (ID) is not defined in a standard way, but the general idea is to estimate the minimum number of dimensions needed to represent the full complexity of a latent space. In our work, we follow the lead of Valeriani et al. [10] and use the TwoNN estimator [39] to compute the IDs. For each $\mathbf{h}_i^l$, it requires the distances $r_{i,1}$ and $r_{i,2}$ to the two closest neighbors in $\mathcal{H}^l$. The ratio $\mu_i = r_{i,2}/r_{i,1}$ thereof follows a Pareto distribution $P(x, \alpha)$ whose shape parameter α is equal to the ID.

The second measure of latent space complexity is *variance@10*, which denotes the portion of variance explained by the first 10 principal components in a principal component analysis.

Third, the Neighborhood Overlap (NO) $\chi_k^{l,m}$ measures how much the k -neighborhoods change between two latent spaces $\mathcal{H}^l$ and $\mathcal{H}^m$ [40]. Let $\mathcal{N}_{k,i}^l$ be the set of the nearest k neighbors of i in $\mathcal{H}^l$, then $\chi_k^{l,m}$ is defined as

$$\chi_k^{l,m} = \frac{1}{N \cdot k} \sum_{i=1}^N |\mathcal{N}_{k,i}^l \cap \mathcal{N}_{k,i}^m|.$$

The higher the values for $\chi_k^{l,l+1}$, the less the compared latent spaces change. Intuitively and empirically demonstrated in this study and [10], the layers of PLMs produce similar latent spaces (high $\chi_k^{l,l+1}$) because each layer adds another layer of abstraction on top of the previous knowledge [17] rather than reorganizing the latent space. Following Valeriani et al., we use $k = 10$ in our experiments and find that χ_k^l is stable across different values of k [10].

4.1.2 Layer probes

Following Alain & Bengio [6], we define linear *layer probes* with weights $\mathbf{w}$ and bias b as:

$$f^l : \mathcal{H}^l \rightarrow \mathbb{L} \quad \text{with} \\ \mathbf{h}_i^l \mapsto \text{act}(\mathbf{w} \cdot \mathbf{h}_i^l + b),$$

where $\mathbb{L}$ is the label space of a dataset: $\mathbb{R}$ for regression tasks and $[0, 1]^D$ for D -dimensional classification problems, act is an activation function, the identity function for regression tasks, a softmax function for binary and multi-target classification problems, and a class-wise sigmoid function for multi-label problems. For amino acid-level tasks, f_l maps from the space of amino acid embeddings $\mathbf{h}_i^l$ and is defined analogously. Since the latent spaces are not necessarily linearly separable, we also use k -nearest neighbor probes operating on $\mathcal{H}^l$ with $k = 10$. k -NNs also align better with the general assumption that similar proteins have similar embeddings.

4.1.3 Sparse layer probes

To estimate how little data is necessary to identify an almost-best layer, we trained sparse layer probes f_{sp}^l on sparse vector spaces $\mathcal{H}_{\text{sp}}^l$ for all layers $l \leq L$ of a PLM. From that, we identify the f_{sp}^{l*} with the best performance on a DT. Then, we compute the ratio of the performance of f^l and f^{l*} , both trained on $\mathcal{H}^l$. This yields the relative-to-best performance metric used in Figure 3E-H.

4.2 Protein Language Models

In this work, we investigate protein language models (PLMs) across five families, spanning a wide range of architectures, training paradigms, model sizes, and embedding dimensions. The ESM-2 [15], ProtT5[31], and ProtST5 [32] models are trained with the *masked language model (MLM) loss*, while ProGen2 [33] and ProtGPT2 [34] were trained with the generative *next-token prediction (NTP) loss*. Due to experimental feasibility and hardware constraints in storing embeddings, we focus only on models with a maximum parameter count of 3 billion. An overview of the technical details of the PLMs is given in Table 2.

Table 2: **Comparison of investigated PLMs.**

[†] ProstT5 is the ProtT5 model finetuned on translating the amino-acid sequences to 3Di sequences and back for 17M structures from the AlphaFold database.

PLM	NLP Arch	Pretraining Datasets	seen data	# layer	# weights [M]	Embed. Dim.
<i>masked-language model loss (bi-directional)</i>						
ESM-2 8M	BERT	UniRef50 & UniRef90	1T AAs	6	12	320
ESM-2 35M	BERT	UniRef50 & UniRef90	1T AAs	12	35	480
ESM-2 150M	BERT	UniRef50 & UniRef90	1T AAs	30	150	640
ESM-2 650M	BERT	UniRef50 & UniRef90	1T AAs	33	650	1280
ESM-2 3B	BERT	UniRef50 & UniRef90	1T AAs	36	3,000	2560
ESMC-300m	Transformer	UniRef70, MGnify 70, JGI 70	6.2T AAs	30	300	960
ESMC-600m	Transformer	UniRef70, MGnify 70, JGI 70	6.2T AAs	36	600	1152
ProtT5	T5	BFD 100, then UniRef50	4.9B + 2B seqs	24	3,000	1024
ProstT5	T5	17M non-redundant AFDB structures [†]	102M + 16.8M seqs	24	3,000	1024
<i>next-token prediction language loss (auto-regressive, generative)</i>						
ProGen2-small	Transformer	UniRef90 & BFD30	175B seqs	12	151	1024
ProGen2-medium	Transformer	UniRef90 & BFD30	175B seqs	27	764	1536
ProGen2-large	Transformer	UniRef90 & BFD30	200B seqs	32	2,700	2560
ProtGPT2	GPT	UniRef50	2.5B seqs	36	738	1280

ESM

The models from the Evolutionary Scale Modeling (ESM) initiative by Facebook AI, later independently as EvolutionaryScale, were the first PLMs to consistently achieve state-of-the-art performance across various tasks [1]. These models are based on the encoder-only BERT architecture [9]. In our work, we use the 8M, 35M, 150M, 650M, and 3B models from the ESM-2 selection, as well as ESMC-300M and ESMC-600M [30]. The ESM-2 models were trained on 1 trillion tokens by first sampling proteins from UniRef50 [45], then, for each sequence, from the corresponding UniRef90 clusters. The ESMC models were trained on 6.2 trillion tokens sampled from a combination of the UniRef, MGnify [46], and JGI databases [47], each clustered at 70% sequence identity.

ProtT5

In the ProtTrans paper, the RostLab tested six different LLM architectures for their transferability to the “protein language”. The pretraining was performed on different combinations of the Big Fantastic Database (BFD100) [48] and UniRef (UniRef100 and UniRef50), clustered at 50% or 100% pairwise sequence identity. The T5-based ProtT5-XL and ProtT5-XXL models emerged as the best. In this work, we use the ProtT5-XL-U50 and will refer to it

as the ProtT5 model. This model was first pretrained on 4.9 billion sequences from the BFD and then finetuned on 2 billion sequences from UniRef50.

ProstT5

The RostLab published another PLM based on their findings in the ProtTrans paper. For ProstT5, they built a dataset from AlphaFoldDB containing pairs of amino acid sequences and 3Di sequences of one protein. The 3Di tokens, computed with FoldSeek, represent the protein structure as a sequence. The authors then fine-tune the ProtT5 model first on the unmasking of both AA and 3Di sequences, and then on the bidirectional translation task between the two formats.

ProGen2

Multiple works show that latent spaces are structured differently for models trained bidirectionally or auto-regressively [10, 11, 12, 13]. To compare the effect of different pretraining objectives, we also include four generative PLMs. The ProGen2 models were trained on the combination of UniRef90 and BFD30 with the next-token prediction loss. Here, we compare only the small, medium, and large models, which have been trained on 175-200 billion sequences. These numbers are much higher than for the bi-directional models because they include many shorter sequences

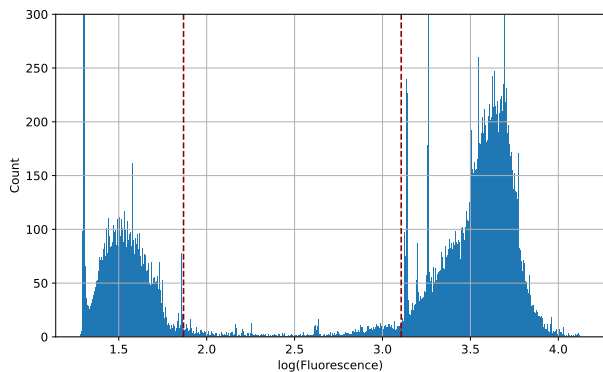


Figure 6: **Label distribution in the Fluorescence dataset.** The blue histogram shows the distribution of log-fluorescence values. The dashed red lines highlight the decision boundaries for the conversion from regression to classification. The data falling in between have been omitted.

to predict early amino acids.

ProtGPT2

An important first step in NLP is to tokenize the input sequences to make it easier to pick up common motifs in the text. For PLMs, subword tokenization is often omitted to generate embeddings for each amino acid. ProtGPT2 is one of the few PLMs that use byte-pair encoding for subword tokenization. BPE computed 50,256 tokens in the Swiss-Prot (version 2021_04), which were then applied to the UniRef50 dataset before pretraining.

4.3 Datasets

The datasets we used to assess the PLMs were originally taken from TAPE [50], with the addition of more recently published Meltome Atlas [24], DeepLoc2.0 [26], and ligand-binding datasets [28]. An overview of key information on the datasets is given in Table 3. We downloaded the Fluorescence, Rocklin stability, DeepSol, Meltome Atlas, and DeepLoc2.0 datasets from the HuggingFace website. The exact vendors are listed in the data availability section.

Fluorescence

Derived from the systematic random mutagenesis of the *Aequorea victoria* green fluorescent protein (GFP) by Sarkisyan et al. [19], this dataset evaluates how amino acid substitutions affect a protein’s biochemical phenotype. It includes approximately 54,000 mutant sequences, each paired with its experimentally measured log-fluorescence intensity. Originally, the task was designed as a regression problem, but the distribution of the log-fluorescence values is bimodal, representing the active and inactive mutants. For that reason, we use this dataset for both classification and regression tasks. The conversion to a classification dataset was performed by fitting a Gaussian Mixture Model to the data, computing 2-standard-deviation intervals around the means, and separating into the upper bound of the lower mode and the lower bound of the upper mode (Figure 6). The data between these two bounds is omitted because, when converting a regression problem to classification, the task is easier to learn when the two classes are separated by an interval rather than by a single threshold.

GB1

This dataset resulted from a deep mutational scan investigating the epistatic effects of mutations at 4 positions in the IgG-binding domain of protein G. It includes ~149,000 mutants and measures the fold change of each mutant relative to the wild-type protein. Similar to the Fluorescence regression task, we predict mutation effects [20].

Rocklin stability

Sourced from the high-throughput screening benchmark developed by Rocklin et al. [21] and curated within the TAPE framework [50], this dataset evaluates the thermodynamic stability of over 69,000 unique short miniproteins (40–50 amino acids in length). The sequence space comprises a diverse mixture of computationally generated *de novo*-designed miniproteins, stable natural control domains (including the Pin1 WW-domain, hYAP65 WW-domain, villin headpiece, and BBL protein), systematically introduced single-residue point mutants, and unfolded negative controls. The associated global regression task requires models to predict a continuous “Stability

Table 3: **Summary of the benchmarked downstream datasets.** For each dataset, we provide structural levels, sizes, and downstream tasks, as well as the mean sequence identity measured as pairwise sequence similarity using MMseqs2. The clusters were computed with CD-HIT at a maximum 40% sequence similarity [49].

Dataset	Level	#Seqs	Mean Length	Associated Downstream Tasks	Mean Seq. Identity (%)	# clusters
Fluorescence	Protein	54,024	237	1. Intensity Regression 2. Active/Dead Classification	0.9671	1
GB1	Protein	149,361	56	3. Binding Fitness Regression	0.8841	1
Rocklin stability	Protein	68,976	45	4. Stability Score Regression	0.2388	11,342
Tsuboyama stability	Protein	117,811	59	5. Stability Regression	0.2192	63
DeepSol	Protein	71,094	295	6. Binary Solubility Classification	0.1653	44,523
SCOPe40 2.08	Protein	15,176	185	7. Fold Classification 8. Superfamily Classification	0.1587	13,901
	Residue			9. DSSP 3-Class Prediction 10. DSSP 8-Class Prediction		
Ligand Binding	Residue	12,317	454	11. Residue-Level Interaction Classification	0.1404	7,540
Meltome Atlas	Protein	30,231	475	12. Species Classification 13. Melting Temperature (T_m) Regression	0.1373	15,579
DeepLoc2.0	Protein	28,302	557	14. Binary Membrane Protein Classification 15. 10-Class Subcellular Localization	0.1366	17,645

Score” derived from multiplexed protease susceptibility assays. This metric quantifies how effectively a surface-displayed protein resists enzymatic degradation by trypsin and chymotrypsin, serving as a direct proxy for folding energetics and structural robustness.

Tsuboyama stability

For this dataset, Tsuboyama et al. measured the thermodynamic folding stability of ~900,000 protein domains (average length: 59 amino acids). The resulting 1.8 million measurements were curated into a set of ~776,000 high-quality folding stabilities, measured as $\Delta\Delta G$, in the original publication, covering all single-mutation variances and some double mutants [22]. In this work, we use the further curated dataset from ProteinGym comprising DMS scans of 117,811 amino acid sequences from 64 species (635 to 5586 mutants per protein) [51].

Meltome Atlas

Similar to the dataset assembled by Rocklin et al. and Tsuboyama et al., the Meltome Atlas measures the

thermostability of various proteins [24]. The main difference lies in the source of proteins: while the dataset compiled by Rocklin et al. consists predominantly of *de novo*-designed short proteins (under 50 amino acids in length), the Meltome Atlas comprises 48,000 natural proteins from 13 different species. The labels are the melting temperatures of proteins in degrees Celsius, and the species are specifically chosen to cover the full range of preferred temperatures, including thermophilic bacteria.

SCOPe40 2.08

This dataset originates from the SCOPe40 dataset, the SCOPe database [27] clustered at 40% sequence similarity. We further filtered it to include only folds or superfamilies with more than 10 samples, removing sparse classes, resulting in ~13,000 experimentally determined protein sequences. The protein-level tasks are the prediction of protein fold and superfamily from the SCOPe40 classification. Additionally, we ran DSSP [29] on the proteins in this dataset, generating 3- and 8-class secondary-structure assignments,

which are predicted at the residue level.

DeepLoc2.0

The DeepLoc2.0 dataset consists of ~28,000 redundancy-reduced, eukaryotic proteins with experimentally verified subcellular localization tags [26]. These comprise a binary classification task distinguishing between soluble and membrane-bound structural forms, and a 10-class multi-class classification task mapping whole proteins to their definitive cellular compartments (such as the nucleus, cytoplasm, or mitochondrion).

DeepSol

Lastly, DeepSol consists of ~71,000 binary protein solubility labels [25]. Khurana et al. preprocessed the dataset to filter out homologous sequences and to remove unwanted bias between the independent test set and the training set.

4.4 Technical requirements

All experiments were conducted on 2 NVIDIA RTX3090 GPUs with 24 GB of RAM, 2 NVIDIA Tesla V100 GPUs with 16 GB of RAM, and the Saarland Informatics Campus High Performance Computing Cluster with multiple NVIDIA A100 GPUs with 40 GB of RAM. All of them used the CUDA v12.9 GPU driver. In total, we consumed ~10 TB of storage space for the protein embeddings and 9.8 TB for the residue embeddings. We did not run the residue-level experiments for the ESM-2 3B, ProGen2-large, and ProtGPT models, but we estimate their memory consumption at an additional 10 TB. On the software side, we relied on PyTorch v2.9.1 and CuML v25.12.0. A full list of dependencies is listed in the requirements.txt file of the GitHub repository linked below.

Data Availability

All code to reproduce the findings of this project is available at [GitHub](#). Additionally, we uploaded all processed datasets to [zenodo](#) [52].

Some of the raw datasets were acquired from the following HuggingFace vendors:

- Fluorescence: [proteinea/fluorescence](#)
- GB1: [SaProtHub/Dataset-GB1-fitness](#)
- Rocklin stability: [proteinglm/stability_prediction](#)
- Meltome Atlas: [cradle-bio/meltome_cluster_split](#)
- DeepLoc2.0: [bloyal/deeploc](#)
- DeepSol: [proteinea/solubility](#)

SCOPe40 was taken directly from the website; the Tsuboyama stability data were extracted from ProteinGym, and the binding data were also sourced directly from the manuscript.

Declarations

Author Contributions: R.J., I.S., and O.V.K. conceived the project. R.J. implemented most of the code and conducted most of the experiments. I.S. conducted the experiments on the fluorescence and stability datasets. A.K. fine-tuned the ESM-2 150M model. D.K. and O.V.K. jointly supervised the project. All authors wrote and revised the manuscript.

Competing Interests: The authors declare no competing interests.

Acknowledgments

We thank Wim Vranken in particular for discussing the Meltome Atlas and for valuable hints and ideas. The idea for this project was first tested in Summer school projects in 2023-2025. The students contributing to the projects' early phase in the three years were Anastasia Lavova, Olga Balakina, Inesa Grigoryan, Igor Chekmarev, Arina Matveichuk, Masia Shchekanova, Veronica Sharonova, Alisa Soloveva, Daniel Korkin, Igor Larin, Semen Karaban, Mariia Sobko, and Evgeniia Samokhvalova. These projects were supervised by R.J., I.S., and A.K. together with Vera Terenteva, Alper Yurtseven, and Guangyi Chen.

References

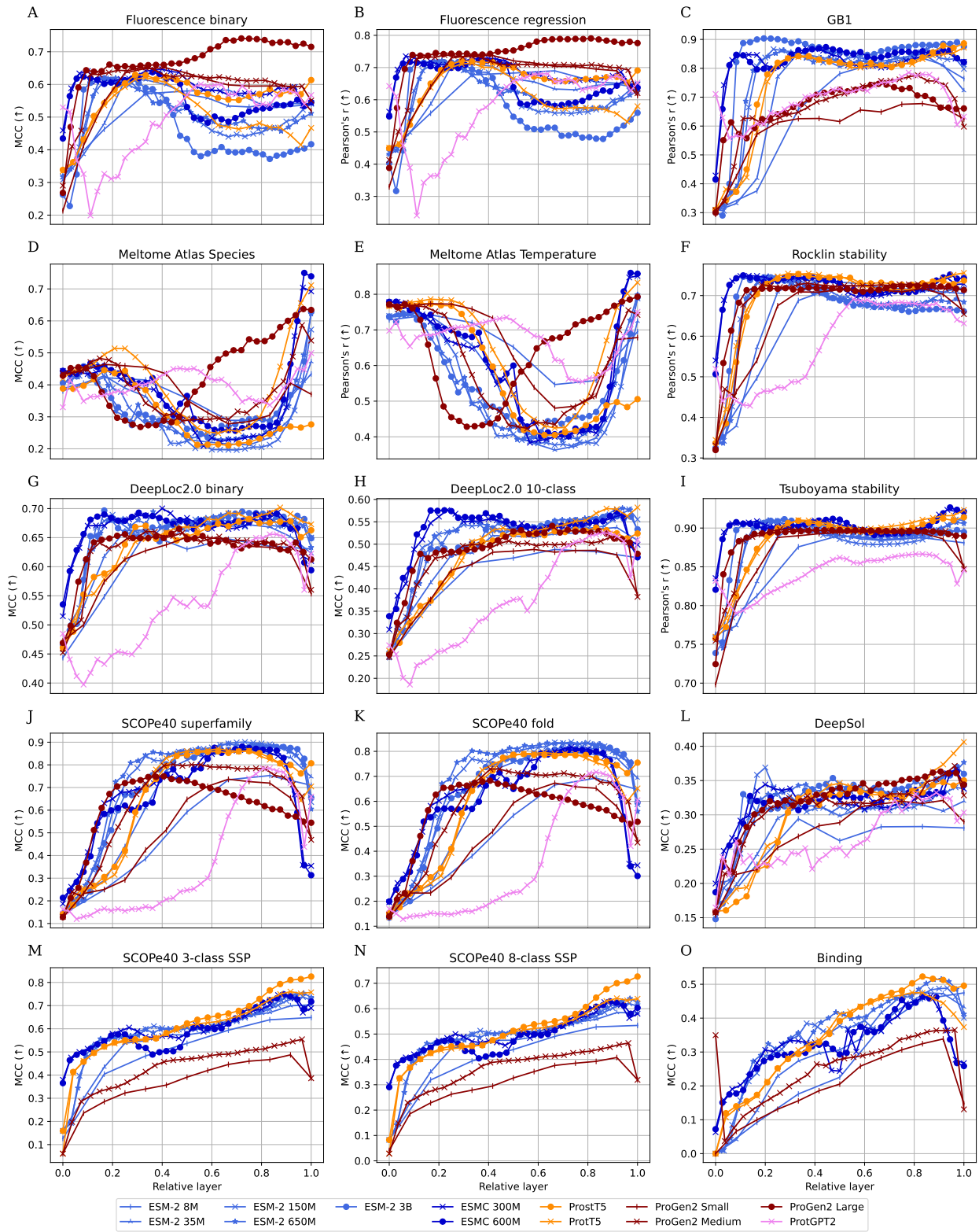
- [1] Alexander Rives, Joshua Meier, Tom Sercu, Sidharth Goyal, Zeming Lin, Jason Liu, Demi Guo, Myle Ott, C Lawrence Zitnick, Jerry

- Ma, et al. Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. *Proceedings of the national academy of sciences*, 118(15):e2016239118, 2021.
- [2] João Capela, Maria Zimmermann-Kogadeeva, Aalt DJ van Dijk, Dick de Ridder, Oscar Dias, and Miguel Rocha. Comparative assessment of protein large language models for enzyme commission number prediction. *BMC bioinformatics*, 26(1):68, 2025.
- [3] Yves Gaetan Nana Teukam, Loïc Kwate Dassi, Matteo Manica, Daniel Probst, Philippe Schwaller, and Teodoro Laino. Language models can identify enzymatic binding sites in protein sequences. *Computational and structural biotechnology journal*, 23:1929–1937, 2024.
- [4] Richard W Shuai, Jeffrey A Ruffolo, and Jeffrey J Gray. Iglm: Infilling language modeling for antibody sequence design. *Cell systems*, 14(11):979–989, 2023.
- [5] Mickael Leclercq and Arnaud Droit. Protein language models: Applications and perspectives. *Journal of Proteome Research*, 25(2):507–524, 2025.
- [6] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644*, 2016.
- [7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [8] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.
- [9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.
- [10] Lucrezia Valeriani, Diego Doimo, Francesca Cuturello, Alessandro Laio, Alessio Ansuini, and Alberto Cazzaniga. The geometry of hidden representations of large transformer models. *Advances in Neural Information Processing Systems*, 36:51234–51252, 2023.
- [11] Jesse Vig, Ali Madani, Lav R Varshney, Caiming Xiong, Richard Socher, and Nazneen Fatema Rajani. Bertology meets biology: Interpreting attention in protein language models. *arXiv preprint arXiv:2006.15222*, 2020.
- [12] Oscar Skea, Md Rifat Arefin, Dan Zhao, Niket Patel, Jalal Naghiyev, Yann LeCun, and Ravid Shwartz-Ziv. Layer by layer: Uncovering hidden representations in language models. *arXiv preprint arXiv:2502.02013*, 2025.
- [13] Matteo Saponati, Pascal Sager, Pau Vilimelis Aceituno, Thilo Stadelmann, and Benjamin Grewe. The underlying structures of self-attention: symmetry, directionality, and emergent dynamics in transformer training. *arXiv preprint arXiv:2502.10927*, 2025.
- [14] Mark Chen, Alec Radford, Rewon Child, Jeffrey Wu, Heewoo Jun, David Luan, and Ilya Sutskever. Generative pretraining from pixels. In *International conference on machine learning*, pages 1691–1703. PMLR, 2020.
- [15] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023.
- [16] Ajit Kumar and IndraPrakash Jha. Layer probing improves kinase functional prediction with protein language models. *arXiv preprint arXiv:2512.00376*, 2025.

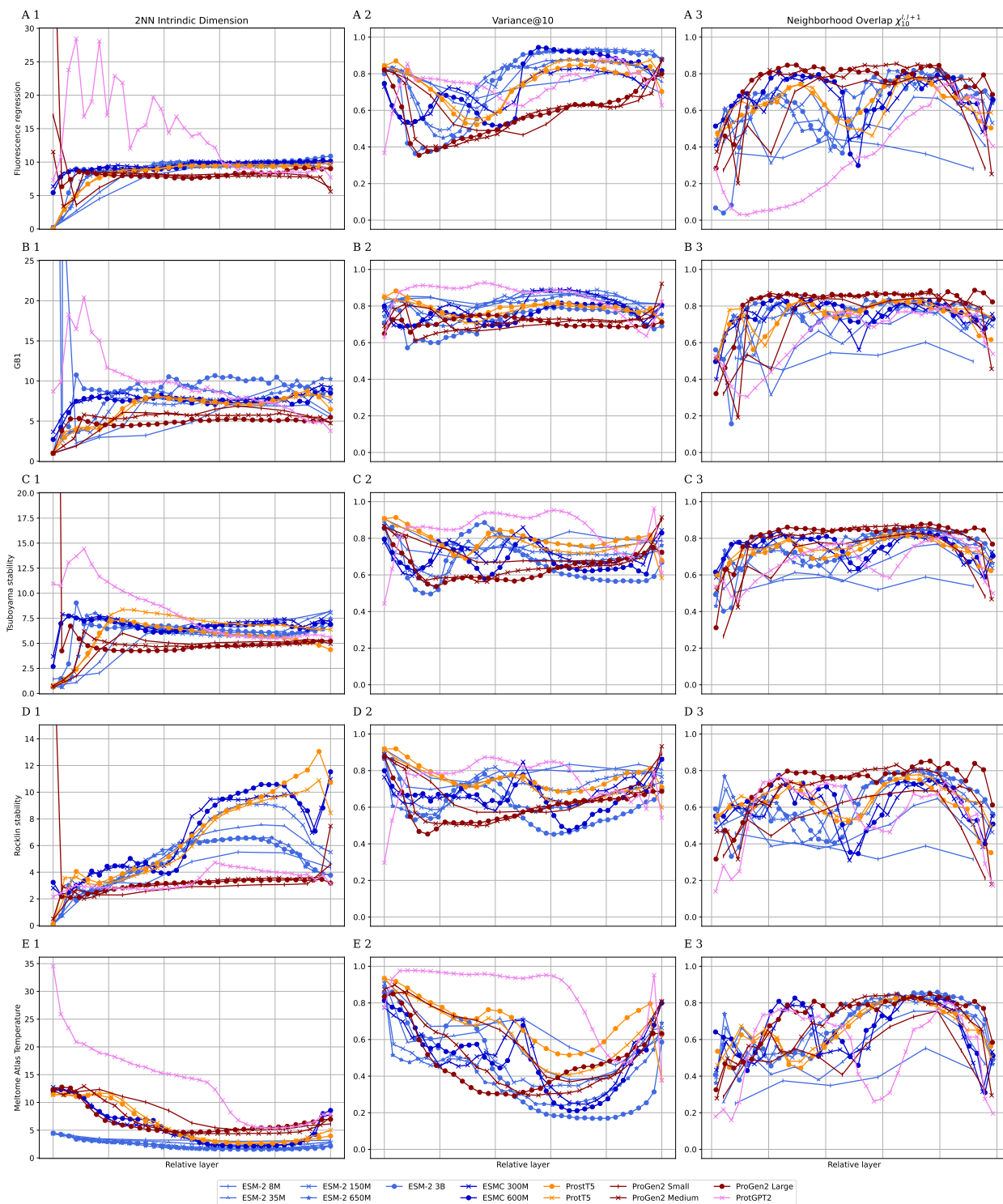
- [17] Matthew D Zeiler and Rob Fergus. Visualizing and understanding convolutional networks. In *European conference on computer vision*, pages 818–833. Springer, 2014.
- [18] R Prabakaran and Yana Bromberg. Quantifying uncertainty in protein representations across models and tasks. *Nature Methods*, pages 1–9, 2026.
- [19] Karen S Sarkisyan, Dmitry A Bolotin, Margarita V Meer, Dinara R Usmanova, Alexander S Mishin, George V Sharonov, Dmitry N Ivankov, Nina G Bozhanova, Mikhail S Baranov, Onuralp Soylemez, et al. Local fitness landscape of the green fluorescent protein. *Nature*, 533(7603):397–401, 2016.
- [20] C Anders Olson, Nicholas C Wu, and Ren Sun. A comprehensive biophysical description of pairwise epistasis throughout an entire protein domain. *Current biology*, 24(22):2643–2651, 2014.
- [21] Gabriel J Rocklin, Tamuka M Chidyausiku, Inna Goreschnik, Alex Ford, Scott Houliston, Alexander Lemak, Lauren Carter, Rashmi Ravichandran, Vikram K Mulligan, Aaron Chevalier, et al. Global analysis of protein folding using massively parallel design, synthesis, and testing. *Science*, 357(6347):168–175, 2017.
- [22] Kotaro Tsuboyama, Justas Dauparas, Jonathan Chen, Elodie Laine, Yasser Mohseni Behbahani, Jonathan J Weinstein, Niall M Mangan, Sergey Ovchinnikov, and Gabriel J Rocklin. Mega-scale experimental analysis of protein folding stability in biology and design. *Nature*, 620(7973):434–444, 2023.
- [23] Ali Madani, Ben Krause, Eric R Greene, Subu Subramanian, Benjamin P Mohr, James M Holton, Jose Luis Olmos Jr, Caiming Xiong, Zachary Z Sun, Richard Socher, et al. Large language models generate functional protein sequences across diverse families. *Nature biotechnology*, 41(8):1099–1106, 2023.
- [24] Anna Jarzab, Nils Kurzawa, Thomas Hopf, Matthias Moerch, Jana Zecha, Niels Leijten, Yangyang Bian, Eva Musiol, Melanie Maschberger, Gabriele Stoeckl, et al. Meltome atlas—thermal proteome stability across the tree of life. *Nature methods*, 17(5):495–503, 2020.
- [25] Sameer Khurana, Reda Rawi, Khalid Kunji, Gwo-Yu Chuang, Halima Bensmail, and Raghvendra Mall. DeepSol: a deep learning framework for sequence-based protein solubility prediction. *Bioinformatics*, 34(15):2605–2613, 2018.
- [26] Vineet Thummuluri, José Juan Almagro Armenteros, Alexander Rosenberg Johansen, Henrik Nielsen, and Ole Winther. DeepLoc 2.0: multi-label subcellular localization prediction using protein language models. *Nucleic acids research*, 50(W1):W228–W234, 2022.
- [27] John-Marc Chandonia, Lindsey Guan, Shiangyi Lin, Changhua Yu, Naomi K Fox, and Steven E Brenner. Scope: improvements to the structural classification of proteins—extended database to facilitate variant interpretation and machine learning. *Nucleic acids research*, 50(D1):D553–D559, 2022.
- [28] Maria Littmann, Michael Heinzinger, Christian Dallago, Konstantin Weissenow, and Burkhard Rost. Protein embeddings and deep learning predict binding residues for various ligand classes. *Scientific reports*, 11(1):23916, 2021.
- [29] Wolfgang Kabsch and Christian Sander. Dictionary of protein secondary structure: pattern recognition of hydrogen-bonded and geometrical features. *Biopolymers: Original Research on Biomolecules*, 22(12):2577–2637, 1983.
- [30] ESM Team. ESM Cambrian: Revealing the mysteries of proteins with unsupervised learning. *EvolutionaryScale Website*, 12 2024.
- [31] Ahmed Elnaggar, Michael Heinzinger, Christian Dallago, Ghalia Rehawi, Yu Wang, Llion Jones, Tom Gibbs, Tamas Feher, Christoph Angerer, Martin Steinegger, et al. ProtTrans: toward understanding the language of life through self-supervised learning. *IEEE transactions on pattern analysis and machine intelligence*, 44(10):7112–7127, 2021.

- [32] Michael Heinzinger, Konstantin Weissenow, Joaquin Gomez Sanchez, Adrian Henkel, Milot Mirdita, Martin Steinegger, and Burkhard Rost. Bilingual language model for protein sequence and structure. *NAR Genomics and Bioinformatics*, 6(4):lqae150, 2024.
- [33] Erik Nijkamp, Jeffrey A Ruffolo, Eli N Weinstein, Nikhil Naik, and Ali Madani. Progen2: exploring the boundaries of protein language models. *Cell systems*, 14(11):968–978, 2023.
- [34] Noelia Ferruz, Steffen Schmidt, and Birte Höcker. Protgpt2 is a deep unsupervised language model for protein design. *Nature communications*, 13(1):4348, 2022.
- [35] Philip Gage. A new algorithm for data compression. *The C Users Journal*, 12(2):23–38, 1994.
- [36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [37] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.
- [38] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training, 2018.
- [39] Elena Facco, Maria d’Errico, Alex Rodriguez, and Alessandro Laio. Estimating the intrinsic dimension of datasets by a minimal neighborhood information. *Scientific reports*, 7(1):12140, 2017.
- [40] Diego Doimo, Aldo Glielmo, Alessio Ansuini, and Alessandro Laio. Hierarchical nucleation in deep neural networks. *Advances in Neural Information Processing Systems*, 33:7526–7536, 2020.
- [41] Karl Pearson. Liii. on lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin philosophical magazine and journal of science*, 2(11):559–572, 1901.
- [42] Sebastián García López, Jesper Salomon, and Wouter Boomsma. Supervised learning of protein melting temperature: Cross-species vs. species-specific prediction. *Proteins: Structure, Function, and Bioinformatics*, 93(12):2158–2166, 2025.
- [43] Roman Joeres, David B Blumenthal, and Olga V Kalinina. Data splitting to avoid information leakage with datasail. *Nature Communications*, 16(1):3337, 2025.
- [44] Ian Tenney, Dipanjan Das, and Ellie Pavlick. Bert rediscovers the classical nlp pipeline. In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pages 4593–4601, 2019.
- [45] Alex Bateman, Maria-Jesus Martin, Sandra Orchard, Michele Magrane, Aduragbemi Adesina, Shadab Ahmad, Emily H Bowler-Barnett, Hema Bye-A-Jee, David Carpentier, Paul Denny, et al. Uniprot: the universal protein knowledgebase in 2025. *Nucleic acids research*, 52(D 1):D609–D617, 2024.
- [46] Lorna Richardson, Ben Allen, Germana Baldi, Martin Beracochea, Maxwell L Bileschi, Tony Burdett, Josephine Burgin, Juan Caballero-Pérez, Guy Cochrane, Lucy J Colwell, et al. Mgnify: the microbiome sequence data analysis resource in 2023. *Nucleic acids research*, 51(D1):D753–D759, 2023.
- [47] Henrik Nordberg, Michael Cantor, Serge Dusheyko, Susan Hua, Alexander Poliakov, Igor Shabalov, Tatyana Smirnova, Igor V Grigoriev, and Inna Dubchak. The genome portal of the department of energy joint genome institute: 2014 updates. *Nucleic acids research*, 42(D1):D26–D31, 2014.
- [48] Martin Steinegger, Milot Mirdita, and Johannes Söding. Protein-level assembly increases protein sequence recovery from metagenomic sam-

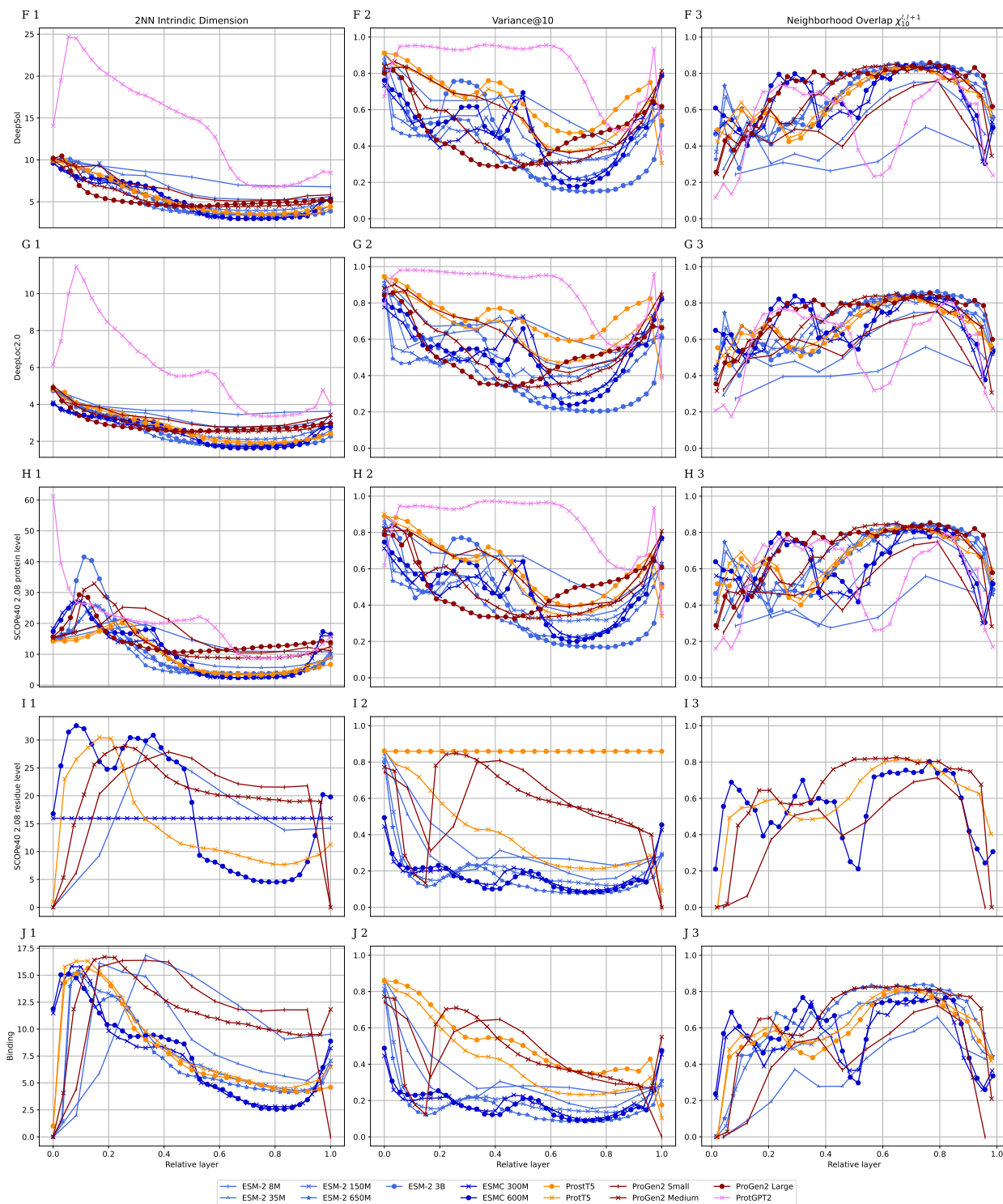
- ples manyfold. *Nature methods*, 16(7):603–606, 2019.
- [49] Weizhong Li and Adam Godzik. Cd-hit: a fast program for clustering and comparing large sets of protein or nucleotide sequences. *Bioinformatics*, 22(13):1658–1659, 2006.
 - [50] Roshan Rao, Nicholas Bhattacharya, Neil Thomas, Yan Duan, Peter Chen, John Canny, Pieter Abbeel, and Yun Song. Evaluating protein transfer learning with tape. *Advances in neural information processing systems*, 32, 2019.
 - [51] Pascal Notin, Aaron Kollasch, Daniel Ritter, Lood Van Niekerk, Steffanie Paul, Han Spinner, Nathan Rollins, Ada Shaw, Rose Orenbuch, Ruben Weitzman, et al. Proteingym: Large-scale benchmarks for protein fitness prediction and design. *Advances in neural information processing systems*, 36:64331–64379, 2023.
 - [52] Roman Joeres, Ilya Senatorov, and Olga V. Kalinina. Task- and Dataset-Specific Information in Protein Language Models, August 12 2026.



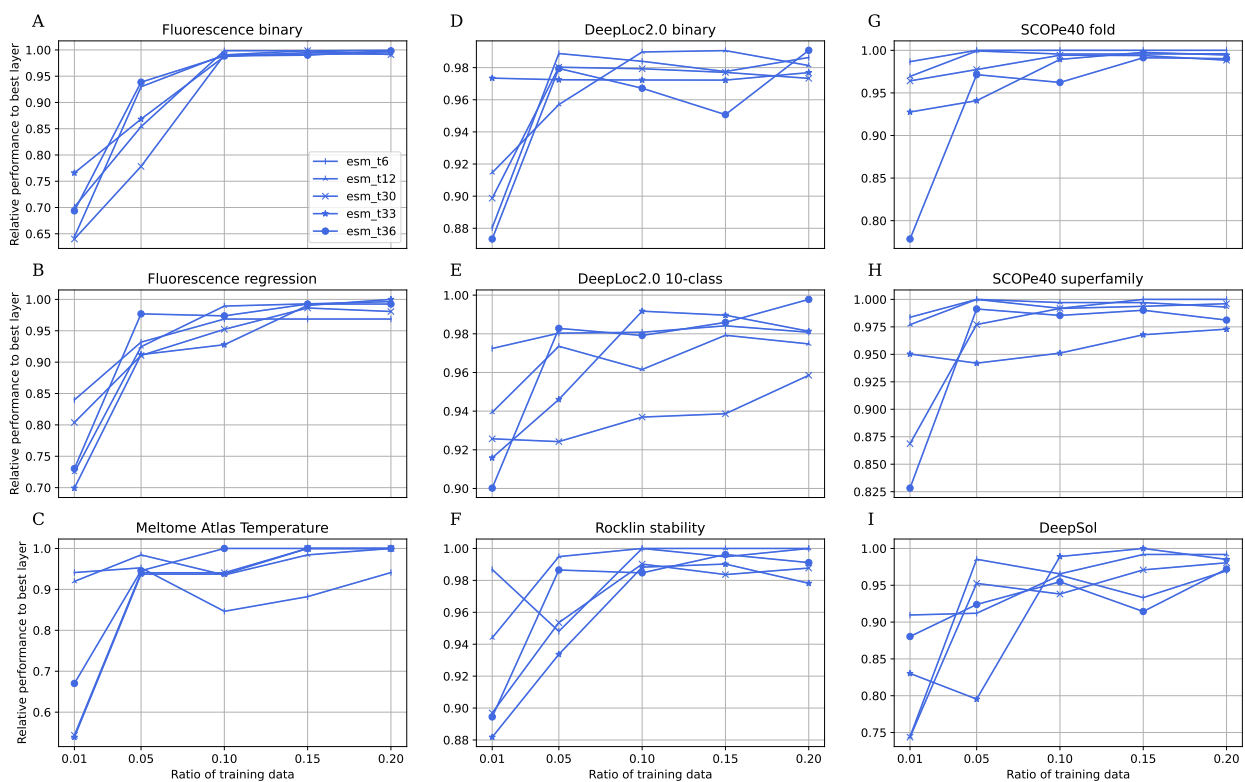
Supplementary Figure 1: **Full layer performances.** Full model resolution of Figure 1 B and C.



Supplementary Figure 2: **Full latent space metrics.** Full model resolution of Figure 2.



Supplementary Figure 3: **Full latent space metrics.** Full model resolution of Figure 2.



Supplementary Figure 4: **Full sparse performance analysis.** Providing additional datasets to the four from the main text.